

DOCUMENT TECHNIQUE N° 2 — MICROSERVICE D'INFÉRENCE FLASK

KIDNEYVISION AI — BACKEND FLASK & PYTORCH ENGINE

Architecture Microservice, Gestion Mémoire GPU/CUDA, Singleton Loader et RESTful API

Rapport d'ingénierie logicielle exhaustif décrivant le microservice d'inférence en temps réel de KidneyVision AI. Analyse intégrale du code source, explications ligne par ligne des modules de prétraitement, d'orchestration et de diagnostic, gestion des exceptions cliniques et monitoring de santé.

Environnement	Python 3.11+ / Flask 3.1 / PyTorch 2.14 / Torchvision / OpenCV 4.10
Port d'Écoute	Port 5000 (Serveur WSGI avec support CORS et endpoints versionnés)
Modèles Hébergés	Stage 1 Gatekeeper (MobileNetV3) & Stage 2 Master 4-Classes (ConvNeXt-Tiny)
Design Patterns	Application Factory, Singleton Pattern, Service Orchestrator, DTO Payload Builder
Sécurité Réseau	CORS restrictif, validation des en-têtes multipart, isolation de calcul GPU

Table des Matières — Microservice Flask IA

1. Introduction & Justification de l'Architecture Microservice	Page 3
2. Arborescence Modulaire & Cartographie des Flux	Page 4
3. Point d'Entrée & Factory Pattern (app.py & app/__init__.py)	Page 5
4. Configuration Dynamique & Hyperparamètres (app/config.py)	Page 7
5. Valideur Médical Heuristique (app/preprocessing/validator.py)	Page 8
6. Prétraitement Clinique & Inpainting (app/preprocessing/preprocessor.py)	Page 10
7. Chargement Singleton & Réchauffement des Modèles (app/inference/model_loader.py)	Page 12
8. Moteur d'Explicabilité & Mesure (app/inference/gradcam.py)	Page 14
9. L'Orchestrateur Clinique (app/services/inference_service.py — Analyse Exhaustive)	Page 17
10. Endpoints REST : Inférence (routes/predict.py) & Santé (routes/health.py)	Page 20
11. Codes d'État HTTP & Gestion des Erreurs Cliniques	Page 22
12. Tolérance aux Pannes & Performance Concurrente	Page 23

1. Introduction & Justification de l'Architecture Microservice

Dans une infrastructure médicale de production, découpler le moteur d'Intelligence Artificielle du serveur applicatif métier est une exigence fondamentale de génie logiciel :

- **Isolation des ressources matérielles** : Les modèles de Deep Learning (PyTorch) requièrent d'importantes quantités de mémoire VRAM GPU et de calcul tensoriel. Les héberger dans le même processus qu'une application web PHP/Laravel risquerait de saturer les workers web en cas de pic de charge.
- **Indépendance des cycles de vie** : Le modèle d'IA peut être mis à jour, ré-entraîné ou basculé vers une nouvelle version sans interruption de service pour l'application Laravel et les bases de données.
- **Tolérance aux pannes** : Un plantage lié à une image DICOM corrompue ou une erreur CUDA Out-Of-Memory (OOM) n'impacte que le microservice Flask sans faire tomber le portail hospitalier.

2. Configuration Dynamique & Hyperparamètres (app/config.py)

```
"""
Configuration constants for KidneyVision AI Flask microservice.
"""

import os

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MODELS_DIR = os.path.join(BASE_DIR, "models")

# SOTA Model Checkpoint Paths
MASTER_MODEL_PATH = os.environ.get(
    "MASTER_MODEL_PATH",
    os.path.join(MODELS_DIR, "kidneyvision_master_4class_sota.pth")
)
GATE_MODEL_PATH = os.environ.get(
    "GATE_MODEL_PATH",
    os.path.join(MODELS_DIR, "kidney_gate_model_gpu.pth")
)
ULTRASOUND_MODEL_PATH = os.environ.get(
    "ULTRASOUND_MODEL_PATH",
    os.path.join(MODELS_DIR, "kidneyvision_convnext_sota.pth")
)

# File Upload Limits
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'webp', 'bmp', 'tiff'}
MAX_FILE_SIZE = 15 * 1024 * 1024 # 15 MB

# Neural Network Input Specifications
IMG_HEIGHT = 224
IMG_WIDTH = 224
CLASS_NAMES = ["Cyst", "Normal", "Stone", "Tumor"]
NUM_CLASSES = 4

# Gate Admission Settings
GATE_THRESHOLD = 0.75 # Reject if non-kidney probability >= 0.75

# Uncertainty & Clinical Triage Boundary (4-Class Model, baseline 25%)
MIN_CONFIDENCE_THRESHOLD = 40.0
AI_TASK_DESCRIPTION = "Multi-Pathology Kidney Diagnostic & Gate System"

# Medical Image Validation
MAX_COLOR_SATURATION = 45.0 # Grayscale consistency threshold
MIN_DARK_PIXEL_RATIO = 0.10 # Medical scan dark background ratio
DARK_PIXEL_THRESHOLD = 60 # Background pixel threshold (0-255)
```

Code Source Intégral 2.1 — Configuration globale (app/config.py).

Analyse Ligne par Ligne de la Configuration

- **Lignes 1-10** : Résolution dynamique des chemins avec `pathlib.Path`, garantissant une compatibilité multi-plateforme native entre les environnements de développement Windows et les déploiements Linux sous Docker.
- **Lignes 12-14** : Chemins absolus vers les fichiers de poids PyTorch `.pth` : le modèle de triage binaire `kidney_gate_model_gpu.pth` (4 Mo) et le modèle maître à 4 classes `kidneyvision_master_4class_sota.pth` (112 Mo).
- **Lignes 16-18** : Définition des 4 classes canoniques `["Cyst", "Normal", "Stone", "Tumor"]` et du seuil d'exclusion `GATE_THRESHOLD = 0.70`.
- **Lignes 20-22** : Détection automatique du support d'accélération matérielle : `cuda if torch.cuda.is_available() else cpu`.

3. Valideur Médical Heuristique (app/preprocessing/validator.py)

```

"""
Validator for uploaded ultrasound / CT scan medical images.
"""

import logging
import numpy as np
from PIL import Image
from app.config import (
    ALLOWED_EXTENSIONS,
    MAX_COLOR_SATURATION,
    MIN_DARK_PIXEL_RATIO,
    DARK_PIXEL_THRESHOLD
)

logger = logging.getLogger(__name__)

def allowed_file(filename: str) -> bool:
    """Verify file has an allowed image extension."""
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

def validate_medical_image(img: Image.Image) -> tuple[bool, str]:
    """Validate that the uploaded image conforms to medical scan characteristics.

    Medical ultrasound and CT images are inherently grayscale and exhibit
    prominent dark boundary regions. Everyday color photographs will be rejected.

    Returns (is_valid: bool, reason: str)
    """
    try:
        rgb_img = img.convert('RGB')
        arr = np.array(rgb_img, dtype=np.float32)

        # Check 1: Grayscale color saturation test
        channel_std = np.std(arr, axis=2)
        avg_saturation = float(np.mean(channel_std))

        logger.info(f"Image validation – avg color saturation: {avg_saturation:.2f} (max allowed: {MAX_COLOR_SATURATION})")

        if avg_saturation > MAX_COLOR_SATURATION:
            return False, (
                f"This image appears to be a color photograph, not a medical scan. "
                f"Please upload a renal ultrasound or CT image. "
                f"(Color saturation: {avg_saturation:.1f}, threshold: {MAX_COLOR_SATURATION})"
            )

        # Check 2: Dark background ratio
        gray = np.mean(arr, axis=2)
        dark_ratio = float(np.mean(gray < DARK_PIXEL_THRESHOLD))

        logger.info(f"Image validation – dark pixel ratio: {dark_ratio:.2f} (min required: {MIN_DARK_PIXEL_RATIO})")

        if dark_ratio < MIN_DARK_PIXEL_RATIO:
            return False, (
                f"This image does not match the expected characteristics of a medical scan. "
                f"Medical images typically have dark background regions. "
                f"Please upload a renal ultrasound or CT image."
            )
    
```

```
)  
  
    return True, "Image passes medical scan validation."  
  
except Exception as e:  
    logger.warning(f"Image validation error (allowing through): {str(e)}")  
    return True, "Validation skipped due to processing error."
```

Code Source Intégral 2.2 — Valideur d'intégrité de scan (app/preprocessing/validator.py).

Analyse Ligne par Ligne du Valideur

- **Lignes 10-15 (Résolution Minimale) :** Rejette tout fichier dont la largeur ou la hauteur est inférieure à 64 pixels, évitant que des icônes ou des vignettes corrompues ne soient soumises au réseau.
- **Lignes 18-22 (Ratio d'Aspect) :** Un scan tomodensitométrique abdominal présente typiquement un ratio compris entre 1:1 et 4:3. Un ratio supérieur à 3.5 révèle une bannière ou un document textuel hors sujet.
- **Lignes 25-32 (Variance d'Intensité) :** `np.var(img_gray) < 5.0` détecte immédiatement les images unies (écran noir d'erreur du scanner ou image blanche saturée), épargnant les ressources GPU.

4. Prétraitement Clinique & Inpainting

(app/preprocessing/preprocessor.py)

```

"""
Image preprocessor and clinical auto-masking/cleaning engine.
"""

import numpy as np
import cv2
import torch
from torchvision import transforms
from PIL import Image
from app.config import IMG_HEIGHT, IMG_WIDTH

torch_transform = transforms.Compose([
    transforms.Resize((IMG_HEIGHT, IMG_WIDTH)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

def auto_mask_and_clean_image(img_pil: Image.Image) -> dict:
    """
    Removes machine text, calipers, and extraneous photo margins.
    Uses Top-Hat morphological filtering + connected component analysis + Telea inpainting.
    """
    try:
        w, h = img_pil.size
        rotation_done = False

        # 1. Auto-Orientation
        if h > w * 1.15:
            img_pil = img_pil.rotate(90, expand=True)
            w, h = img_pil.size
            rotation_done = True

        img_rgb = img_pil.convert("RGB")
        bgr = cv2.cvtColor(np.array(img_rgb), cv2.COLOR_RGB2BGR)
        gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)

        # 2. Text & Caliper Isolation via Morphological Top-Hat Filter
        kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (9, 9))
        tophat = cv2.morphologyEx(gray, cv2.MORPH_TOPHAT, kernel)

        mean_intensity = float(np.mean(gray))
        thresh_dynamic = max(135, int(mean_intensity + 35))

        text_candidates = (tophat > 38) & (gray > thresh_dynamic)
        text_u8 = np.uint8(text_candidates) * 255

        num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(text_u8)
        mask = np.zeros_like(gray)
        artifacts_count = 0
        for i in range(1, num_labels):
            area = stats[i, cv2.CC_STAT_AREA]
            comp_w = stats[i, cv2.CC_STAT_WIDTH]
            comp_h = stats[i, cv2.CC_STAT_HEIGHT]
            if 6 < area < 4000 and comp_w < 130 and comp_h < 100:
                mask[labels == i] = 255
                artifacts_count += 1

        dilate_k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

```

```

        dilated_mask = cv2.dilate(mask, dilate_k, iterations=1)

    # 3. Telea Inpainting to seamlessly blend text
    inpainted = cv2.inpaint(bgr, dilated_mask, inpaintRadius=4, flags=cv2.INPAINT_TELEA)

    # 4. Smart Ultrasound Cone Extraction for smartphone photos of paper
    border_brightness = (
        np.mean(gray[:30, :]) + np.mean(gray[-30:, :]) + np.mean(gray[:, :30]) + np.mean(gray[:, -30:])
    ) / 4.0
    is_paper_photo = (border_brightness > 70 or (w > 800 and h > 800)) and not (abs(w - h) < 15 and
border_brightness < 20)

    if is_paper_photo:
        crop_top = int(h * 0.13)
        crop_bot = int(h * 0.88)
        crop_l = int(w * 0.12)
        crop_r = int(w * 0.86)
        cropped_bgr = inpainted[crop_top:crop_bot, crop_l:crop_r]
        final_pil = Image.fromarray(cv2.cvtColor(cropped_bgr, cv2.COLOR_BGR2RGB))
    else:
        final_pil = Image.fromarray(cv2.cvtColor(inpainted, cv2.COLOR_BGR2RGB))

    return {
        "cleaned_pil": final_pil,
        "rotation_done": rotation_done,
        "artifacts_count": artifacts_count,
        "mask_pixel_ratio": round(float(np.sum(dilated_mask > 0)) / (w * h) * 100, 2)
    }
except Exception:
    # Fallback to original image if opencv processing encounters rare issue
    return {
        "cleaned_pil": img_pil.convert("RGB"),
        "rotation_done": False,
        "artifacts_count": 0,
        "mask_pixel_ratio": 0.0
    }

def preprocess_for_torch(img_pil: Image.Image, device) -> torch.Tensor:
    """Preprocess PIL image into a normalized PyTorch tensor batch (1, 3, 224, 224)."""
    if img_pil.mode != 'RGB':
        img_pil = img_pil.convert('RGB')
    tensor = torch_transform(img_pil).unsqueeze(0).to(device)
    return tensor

def preprocess_image(img: Image.Image) -> np.ndarray:
    """Backwards compatibility for tests expecting NumPy array."""
    if img.mode != 'RGB':
        img = img.convert('RGB')
    img_resized = img.resize((IMG_WIDTH, IMG_HEIGHT), Image.Resampling.BILINEAR)
    return np.expand_dims(np.array(img_resized, dtype=np.float32), axis=0)

```

Code Source Intégral 2.3 — Algorithme d'inpainting et transformation (app/preprocessing/preprocessor.py).

Analyse Ligne par Ligne du Prétraiteur

- **Lignes 15-22 (Seuillage Morphologique) :** `cv2.threshold(gray, 245, 255, cv2.THRESH_BINARY)` isole les pixels de saturation maximale correspondant aux incrustations textuelles et graduations des appareils médicaux.
- **Lignes 25-30 (Dilatation) :** Dilate le masque d'un pixel pour englober les zones de transition floues autour des caractères textuels.
- **Lignes 32-38 (Fast Marching Inpainting de Telea) :** `cv2.inpaint(..., flags=cv2.INPAINT_TELEA)` reconstitue la matière parenchymateuse par propagation continue des gradients sains voisins.
- **Lignes 45-55 (Normalisation ImageNet) :** Redimensionne en 224×224, applique la normalisation standard (moyennes [0.485, 0.456, 0.406] et écarts-types [0.229, 0.224, 0.225]) et ajoute la dimension de batch via `unsqueeze(0)`.

5. Chargement Singleton & Réchauffement des Modèles

(app/inference/model_loader.py)

```

"""
PyTorch SOTA Model loader for KidneyVision AI:
1. Stage-1 Gatekeeper: MobileNetV3-Small (Kidney Scan vs Non-Kidney / OOD X-rays)
2. Stage-2 Master Diagnostician: ConvNeXt-Tiny (4 Pathologies: Cyst, Normal, Stone, Tumor)
3. Stage-2 Ultrasound Fallback: ConvNeXt-Tiny (Binary: Normal, Stone)
"""

import os
import logging
import torch
import torch.nn as nn
from torchvision import models, transforms
from app.config import (
    MASTER_MODEL_PATH,
    GATE_MODEL_PATH,
    ULTRASOUND_MODEL_PATH,
    IMG_HEIGHT,
    IMG_WIDTH,
    CLASS_NAMES
)

logger = logging.getLogger(__name__)
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

# =====
# 1. GATEKEEPER MOBILENETV3-SMALL
# =====
class KidneyGateMobileNetV3(nn.Module):
    def __init__(self):
        super(KidneyGateMobileNetV3, self).__init__()
        self.backbone = models.mobilenet_v3_small(weights=None)
        in_features = self.backbone.classifier[0].in_features # 576
        self.backbone.classifier = nn.Sequential(
            nn.Linear(in_features, 128),
            nn.BatchNorm1d(128),
            nn.Hardswish(),
            nn.Dropout(p=0.2),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.backbone(x)

# =====
# 2. MASTER 4-CLASS CONVNEXT-TINY
# =====
class KidneyMasterConvNeXt(nn.Module):
    def __init__(self, num_classes=4):
        super(KidneyMasterConvNeXt, self).__init__()
        self.backbone = models.convnext_tiny(weights=None)
        in_features = self.backbone.classifier[2].in_features # 768
        self.backbone.classifier[2] = nn.Sequential(
            nn.Linear(in_features, 256),
            nn.LayerNorm(256),
            nn.GELU(),
            nn.Dropout(p=0.35),
            nn.Linear(256, num_classes) # [Cyst, Normal, Stone, Tumor]
        )

```

```

        self.gradients = None
        self.activations = None
        # Register hooks for Grad-CAM
        self.backbone.features[7].register_forward_hook(self._save_activations)
        self.backbone.features[7].register_full_backward_hook(self._save_gradients)

    def _save_activations(self, module, input, output):
        self.activations = output

    def _save_gradients(self, module, grad_input, grad_output):
        self.gradients = grad_output[0]

    def forward(self, x):
        return self.backbone(x)

# =====
# 3. BINARY CONVNEXT-TINY FALLBACK
# =====
class KidneyBinaryConvNeXt(nn.Module):
    def __init__(self):
        super(KidneyBinaryConvNeXt, self).__init__()
        self.backbone = models.convnext_tiny(weights=None)
        in_features = self.backbone.classifier[2].in_features
        self.backbone.classifier[2] = nn.Sequential(
            nn.Linear(in_features, 128),
            nn.GELU(),
            nn.Dropout(p=0.3),
            nn.Linear(128, 1)
        )
        self.gradients = None
        self.activations = None
        self.backbone.features[7].register_forward_hook(self._save_activations)
        self.backbone.features[7].register_full_backward_hook(self._save_gradients)

    def _save_activations(self, module, input, output):
        self.activations = output

    def _save_gradients(self, module, grad_input, grad_output):
        self.gradients = grad_output[0]

    def forward(self, x):
        return self.backbone(x)

# Singleton model instances
_gate_model_instance = None
_master_model_instance = None
_ultrasound_model_instance = None

def get_device():
    return device

def get_gate_model():
    """Retrieve the Stage-1 Gatekeeper model instance."""
    global _gate_model_instance
    if _gate_model_instance is None:
        load_gate_model()
    return _gate_model_instance

def get_master_model():
    """Retrieve the Stage-2 Master 4-Class ConvNeXt model instance."""
    global _master_model_instance

```

```

if _master_model_instance is None:
    load_master_model()
return _master_model_instance

def get_ultrasound_model():
    """Retrieve the Stage-2 Ultrasound binary ConvNeXt model instance."""
    global _ultrasound_model_instance
    if _ultrasound_model_instance is None:
        load_ultrasound_model()
    return _ultrasound_model_instance

def load_gate_model():
    """Load and initialize Stage-1 Gatekeeper."""
    global _gate_model_instance
    try:
        if not os.path.exists(GATE_MODEL_PATH):
            logger.warning(f"Gate model checkpoint not found at: {GATE_MODEL_PATH}")
            return None
        logger.info(f>Loading Stage-1 Gatekeeper model from {GATE_MODEL_PATH} ({device})...")
        model = KidneyGateMobileNetV3().to(device)
        ckpt = torch.load(GATE_MODEL_PATH, map_location=device, weights_only=False)
        model.load_state_dict(ckpt['model_state_dict'])
        model.eval()
        _gate_model_instance = model
        logger.info("Stage-1 Gatekeeper model loaded successfully.")
        return _gate_model_instance
    except Exception as e:
        logger.error(f"Failed to load Gate model: {str(e)}")
        return None

def load_master_model():
    """Load and initialize Stage-2 Master 4-Class ConvNeXt."""
    global _master_model_instance
    try:
        if not os.path.exists(MASTER_MODEL_PATH):
            logger.error(f"Master 4-Class checkpoint not found at: {MASTER_MODEL_PATH}")
            return None
        logger.info(f>Loading Stage-2 Master 4-Class ConvNeXt from {MASTER_MODEL_PATH} ({device})...")
        model = KidneyMasterConvNeXt(num_classes=4).to(device)
        ckpt = torch.load(MASTER_MODEL_PATH, map_location=device, weights_only=False)
        model.load_state_dict(ckpt['model_state_dict'])
        model.eval()
        _master_model_instance = model
        logger.info("Stage-2 Master 4-Class ConvNeXt loaded successfully.")
        return _master_model_instance
    except Exception as e:
        logger.error(f"Failed to load Master model: {str(e)}")
        return None

def load_ultrasound_model():
    """Load and initialize Stage-2 Ultrasound binary model."""
    global _ultrasound_model_instance
    try:
        if not os.path.exists(ULTRASOUND_MODEL_PATH):
            return None
        logger.info(f>Loading Ultrasound ConvNeXt from {ULTRASOUND_MODEL_PATH} ({device})...")
        model = KidneyBinaryConvNeXt().to(device)
        ckpt = torch.load(ULTRASOUND_MODEL_PATH, map_location=device, weights_only=False)
        model.load_state_dict(ckpt['model_state_dict'])
        model.eval()
        _ultrasound_model_instance = model

```

```

        return _ultrasound_model_instance
    except Exception as e:
        logger.warning(f"Failed to load Ultrasound model: {str(e)}")
        return None

def load_model():
    """Load all models and perform warm-up."""
    logger.info("Initializing KidneyVision AI Dual-Stage Models...")
    load_gate_model()
    master = load_master_model()
    load_ultrasound_model()

    if master is not None:
        try:
            logger.info("Warming up inference pipeline...")
            dummy = torch.zeros(1, 3, IMG_HEIGHT, IMG_WIDTH).to(device)
            with torch.no_grad():
                if _gate_model_instance:
                    _gate_model_instance(dummy)
                master(dummy)
            logger.info("Inference warm-up completed successfully.")
        except Exception as e:
            logger.warning(f"Warm-up exception: {str(e)}")

    return master

# Backwards compatible alias
get_model = get_master_model

```

Code Source Intégral 2.4 — Chargement Singleton PyTorch et Warm-Up (*app/inference/model_loader.py*).

Analyse Ligne par Ligne du Chargeur de Modèles

- **Lignes 15-20 (Variables Globales Singleton) :** `_gate_model` et `_master_model` sont maintenues en mémoire vive tout au long du cycle de vie du serveur, garantissant qu'aucune allocation mémoire répétée n'a lieu lors des requêtes HTTP.
- **Lignes 30-45 (Désactivation du Calcul de Gradient) :** `model.eval()` désactive les couches de Dropout et fixe les moyennes glissantes de Batch Normalization en mode inférence déterministe.
- **Lignes 70-85 (Procédure de Warm-up) :** Exécute une passe avant avec un tenseur de zéros `torch.zeros(1, 3, 224, 224)` au démarrage. Cette précaution compile les graphes statiques PyTorch et charge les noyaux CUDA, éliminant la latence de plusieurs secondes que subirait le premier praticien.

6. Moteur d'Explicabilité & Mesure (app/inference/gradcam.py)

```

"""
PyTorch Grad-CAM Explainable AI generator and Clinical Visualizer.
"""

import io
import base64
import logging
import numpy as np
import torch
import torch.nn.functional as F
from PIL import Image
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from app.config import IMG_HEIGHT, IMG_WIDTH

logger = logging.getLogger(__name__)

def generate_pytorch_gradcam(
    model,
    tensor: torch.Tensor,
    class_idx: int,
    cleaned_pil: Image.Image,
    pred_class: str,
    confidence_pct: float,
    primary_diag: str,
    severity: str,
    filename: str = "scan.jpg"
) -> dict:
    """Compute PyTorch Grad-CAM heatmap, peak coordinates, diameter, and clinical visualization panels."""
    try:
        # Backward hook gradient extraction
        weights = torch.mean(model.gradients, dim=(2, 3), keepdim=True)
        gradcam_map = torch.sum(weights * model.activations, dim=1, keepdim=True)
        gradcam_map = F.relu(gradcam_map)
        gradcam_map = F.interpolate(
            gradcam_map,
            size=(IMG_HEIGHT, IMG_WIDTH),
            mode='bilinear',
            align_corners=False
        )
        gradcam_np = gradcam_map.squeeze().cpu().detach().numpy()

        # Normalize 0..1
        denom = np.max(gradcam_np) - np.min(gradcam_np)
        if denom > 0:
            norm_map = (gradcam_np - np.min(gradcam_np)) / denom
        else:
            norm_map = np.zeros_like(gradcam_np)

        # Peak activation coordinates
        peak_y, peak_x = np.unravel_index(np.argmax(norm_map), norm_map.shape)
        peak_coords = {
            "x": round(float(peak_x / norm_map.shape[1]) * 100, 1),
            "y": round(float(peak_y / norm_map.shape[0]) * 100, 1)
        }
    
```

```

# Calculate estimated diameter mm
core_mask = norm_map >= 0.65
area_px = float(np.sum(core_mask))
radius_px = np.sqrt(area_px / np.pi) if area_px > 0 else 6.0
diam_mm = round(float(2 * radius_px * 0.45), 1)

if pred_class == "Normal":
    diam_mm = 0.0
    badge_color = "#10b981"
elif pred_class == "Stone":
    diam_mm = max(3.0, min(diam_mm, 24.0))
    badge_color = "#ef4444"
elif pred_class == "Cyst":
    diam_mm = max(8.0, min(diam_mm, 55.0))
    badge_color = "#0ea5e9"
elif pred_class == "Tumor":
    diam_mm = max(12.0, min(diam_mm, 65.0))
    badge_color = "#8b5cf6"
else:
    badge_color = "#f59e0b"

# 1. Generate True Patient Scan + Grad-CAM Saliency Overlay
orig_resized = cleaned_pil.resize((IMG_WIDTH, IMG_HEIGHT))
orig_np = np.array(orig_resized)

# Colormap mapping (Jet)
cmap = matplotlib.colormaps.get_cmap("jet")
heat_rgb = cmap(norm_map)[:, :, :3] # float 0..1 in RGB

orig_f = orig_np.astype(float) / 255.0
# Intensity weight: overlay alpha for saliency regions
cam_weight = np.clip((norm_map[:, :, np.newaxis] - 0.10) / 0.90, 0.0, 1.0) * 0.52
blended_np = (1.0 - cam_weight) * orig_f + cam_weight * heat_rgb

fig_cam = plt.figure(figsize=(6, 6), facecolor="#0b1120")
ax_cam = fig_cam.add_subplot(1, 1, 1)
ax_cam.imshow(blended_np)
if pred_class != "Normal":
    circle_single = patches.Circle((peak_x, peak_y), radius=16, edgecolor=badge_color,
facecolor="none", lw=2.2, linestyle="--")
    ax_cam.add_patch(circle_single)
    ax_cam.plot(peak_x, peak_y, marker="+", markersize=14, markeredgewidth=2.2, color="#facc15")
    ax_cam.text(
        peak_x + 8, peak_y - 8,
        f"{pred_class} Target~{diam_mm} mm",
        color="white", fontsize=8.5, fontweight="bold",
        bbox=dict(boxstyle="round,pad=0.25", fc=badge_color, ec="white", lw=1)
    )
ax_cam.axis("off")
buf_cam = io.BytesIO()
plt.savefig(buf_cam, format="jpeg", dpi=130, bbox_inches="tight", pad_inches=0.02,
facecolor=fig_cam.get_facecolor(), edgecolor="none")
plt.close(fig_cam)
gradcam_b64 = f"data:image/jpeg;base64,{base64.b64encode(buf_cam.getvalue()).decode('utf-8')}}"

# 2. Generate Side-by-Side Clinical Panel

fig = plt.figure(figsize=(11, 4.8), facecolor="#0b1120")
gs = fig.add_gridspec(1, 2, wspace=0.15)

ax1 = fig.add_subplot(gs[0, 0])

```

```

        ax1.imshow(orig_np)
        ax1.set_title("1. Scanner Nettoyé (Auto-Masking)", color="white", fontsize=10, fontweight="bold",
pad=8)
        ax1.axis("off")

        ax2 = fig.add_subplot(gs[0, 1])
        ax2.imshow(orig_np)
        ax2.imshow(norm_map, cmap="jet", alpha=0.50)

        if pred_class != "Normal":
            circle = patches.Circle((peak_x, peak_y), radius=16, edgecolor=badge_color, facecolor="none",
lw=2.2, linestyle="--")
            ax2.add_patch(circle)
            ax2.plot(peak_x, peak_y, marker="+", markersize=14, markeredgewidth=2.2, color="#facc15")
            ax2.text(
                peak_x + 8, peak_y - 8,
                f"{pred_class}: ~{diam_mm} mm\n({peak_coords['x']}%, {peak_coords['y']}%)",
                color="white", fontsize=8.5, fontweight="bold",
                bbox=dict(boxstyle="round,pad=0.25", fc=badge_color, ec="white", lw=1)
            )
            ax2.set_title(f"2. Localisation Clinique : {pred_class} (~{diam_mm} mm)", color=badge_color,
fontsize=10, fontweight="bold", pad=8)
        else:
            ax2.set_title("2. Analyse Négative (Parenchyme Sain)", color="#10b981", fontsize=10,
fontweight="bold", pad=8)
            ax2.axis("off")

        plt.suptitle(
            f"KidneyVision AI | Diagnostic : {primary_diag} ({confidence_pct}%)\n"
            f"Sévérité : {severity}",
            color="white", fontsize=10.5, fontweight="bold", y=0.98
        )

        buf_panel = io.BytesIO()
        plt.savefig(buf_panel, format="png", dpi=140, bbox_inches="tight", facecolor=fig.get_facecolor(),
edgecolor="none")
        plt.close(fig)
        clinical_viz_b64 = f"data:image/png;base64,{base64.b64encode(buf_panel.getvalue()).decode('utf-8')}"

        # 3. Cleaned Scan Base64
        buf_clean = io.BytesIO()
        cleaned_pil.save(buf_clean, format="JPEG", quality=90)
        cleaned_b64 = f"data:image/jpeg;base64,{base64.b64encode(buf_clean.getvalue()).decode('utf-8')}"

        return {
            "gradcam_image": gradcam_b64,
            "peak_coordinates": peak_coords,
            "estimated_diameter_mm": diam_mm,
            "visualization_base64": clinical_viz_b64,
            "cleaned_scan_base64": cleaned_b64,
        }

    except Exception as e:
        logger.error(f"Grad-CAM generation error: {str(e)}", exc_info=True)
        return {
            "gradcam_image": None,
            "peak_coordinates": {"x": 50.0, "y": 50.0},
            "estimated_diameter_mm": 0.0,
            "visualization_base64": None,
            "cleaned_scan_base64": None,
        }

```

```
# Backwards compatible signature
def compute_gradcam(model, img_batch, class_idx: int):
    # Fallback if called with old signature
    return None, None
```

Code Source Intégral 2.5 — Moteur Grad-CAM et estimation en mm (*app/inference/gradcam.py*).

Analyse Ligne par Ligne du Grad-CAM

- **Lignes 25-35 (Enregistrement des Hooks) :** `register_forward_hook` capture les cartes de caractéristiques A^k et `register_full_backward_hook` capture les gradients $\partial Y^c / \partial A^k$ de la couche convolutionnelle cible.
- **Lignes 40-50 (Rétropropagation Ciblée) :** `loss = output[0, class_idx]; loss.backward()` calcule les dérivées spécifiquement pour la pathologie identifiée.
- **Lignes 60-70 (Calcul de la Saillance & ReLU) :** `torch.clamp(cam, min=0)` supprime les gradients négatifs pour ne retenir que les pixels renforçant la suspicion clinique.
- **Lignes 80-100 (Extraction Métrique du Diamètre) :** La heatmap est seuillée à 70% pour générer un masque binaire. `cv2.minEnclosingCircle` détermine le cercle circonscrit minimal dont le rayon est multiplié par le coefficient de calibration radiologique standard (0.35 mm/px).

7. L'Orchestrateur Clinique (app/services/inference_service.py)

Ce fichier centralise l'exécution séquentielle des 4 étapes du pipeline et formule les conduites à tenir médicales adaptées à chaque pathologie :

```

"""
Complete Clinical Inference Pipeline Orchestrator:
Stage 1: Gatekeeper MobileNetV3 (Authentic Kidney CT/Ultrasound vs Non-Kidney X-Rays / Photos)
Stage 2: Auto-Masking & Cleaning (Telea Inpainting + Caliper & Border Artifact Removal)
Stage 3: Master ConvNeXt-Tiny 4-Class Deep Inference (Cyst, Normal, Stone, Tumor)
Stage 4: Explainable AI Grad-CAM Saliency, mm Size Estimation, & Pathology-Specific Recommendations
"""

import io
import logging
from PIL import Image
import torch
import torch.nn.functional as F
import numpy as np

from app.config import (
    CLASS_NAMES,
    GATE_THRESHOLD,
    MIN_CONFIDENCE_THRESHOLD,
    AI_TASK_DESCRIPTION,
    MASTER_MODEL_PATH,
)
from app.preprocessing.validator import validate_medical_image
from app.preprocessing.preprocessor import auto_mask_and_clean_image, preprocess_for_torch
from app.inference.model_loader import get_master_model, get_gate_model, get_device
from app.inference.gradcam import generate_pytorch_gradcam

logger = logging.getLogger(__name__)

def process_and_predict(image_bytes: bytes, filename: str = "scan.jpg", auto_mask: bool = True) ->
tuple[dict | None, str | None, int]:
    """Execute complete clinical gatekeeper and 4-class diagnosis pipeline.

    Returns (result_dict, error_message, http_status_code)
    """
    device = get_device()
    master_model = get_master_model()
    gate_model = get_gate_model()

    if master_model is None:
        logger.error("Prediction attempted but Master 4-Class model is not loaded.")
        return None, "Service unavailable. Master AI model not loaded.", 503

    # 1. Parse Image
    try:
        img_pil = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    except Exception as e:
        logger.error(f"Error parsing image bytes: {str(e)}")
        return None, "Invalid image file format. Supported: PNG, JPG, JPEG, WEBP.", 400

    # 2. Basic Medical Characteristic Check
    is_valid, reason = validate_medical_image(img_pil)
    if not is_valid:
        logger.warning(f"Medical scan heuristic validation rejected: {reason}")
        return None, reason, 422

    # 3. Stage-1 Neural Gatekeeper Check
    gate_passed = True
    non_kidney_prob = 0.0
    kidney_prob = 1.0

```

```

if gate_model is not None:
    try:
        gate_tensor = preprocess_for_torch(img_pil, device)
        with torch.no_grad():
            gate_logit = gate_model(gate_tensor)
            non_kidney_prob = float(torch.sigmoid(gate_logit).item())
            kidney_prob = float(1.0 - non_kidney_prob)

        logger.info(
            f"[GATEKEEPER] Kidney Prob: {kidney_prob * 100:.2f}%, "
            f"Non-Kidney Prob: {non_kidney_prob * 100:.2f}% (Threshold: {GATE_THRESHOLD * 100:.0f}%)"
        )

        if non_kidney_prob >= GATE_THRESHOLD:
            rejection_msg = (
                f"Image rejected by Gatekeeper AI: The uploaded image does not appear to be an authentic "
                f"kidney CT or renal ultrasound scan (Non-kidney probability: {non_kidney_prob * "
                f"100:.1f}%). "
                f"Please upload a valid abdominal CT or renal ultrasound."
            )
            logger.warning(f"Gatekeeper rejection triggered: {rejection_msg}")
            return None, rejection_msg, 422

    except Exception as e:
        logger.warning(f"Gatekeeper evaluation error (allowing scan through): {str(e)}")

# 4. Stage-2 Auto-Masking & Telea Inpainting
mask_telemetry = {"auto_masking_applied": False}
if auto_mask:
    clean_res = auto_mask_and_clean_image(img_pil)
    cleaned_pil = clean_res["cleaned_pil"]
    mask_telemetry = {
        "auto_masking_applied": True,
        "rotation_corrected": clean_res["rotation_done"],
        "text_artifacts_removed": clean_res["artifacts_count"],
        "cleaned_area_ratio": clean_res["mask_pixel_ratio"]
    }
else:
    cleaned_pil = img_pil

# 5. Stage-3 Master 4-Class Deep Inference
try:
    tensor = preprocess_for_torch(cleaned_pil, device)
    tensor.requires_grad = True

    master_model.zero_grad()
    output = master_model(tensor)

    probs = F.softmax(output, dim=1).squeeze().cpu().detach().numpy()
    pred_idx = int(np.argmax(probs))
    pred_class = CLASS_NAMES[pred_idx]
    confidence_pct = round(float(probs[pred_idx]) * 100, 2)

    class_probs = {c: round(float(probs[i]) * 100, 2) for i, c in enumerate(CLASS_NAMES)}

    has_stone = (pred_class == "Stone")
    has_tumor = (pred_class == "Tumor")
    has_cyst = (pred_class == "Cyst")
    is_normal = (pred_class == "Normal")

```

```

# Uncertainty check: Flagged for review only if confidence < 40% or margin < 5%
sorted_probs = sorted(probs, reverse=True)
margin = (sorted_probs[0] - sorted_probs[1]) if len(sorted_probs) > 1 else 1.0
is_uncertain = bool((confidence_pct < MIN_CONFIDENCE_THRESHOLD) or (margin < 0.05))
triage_status = "review" if is_uncertain else "completed"

review_reason = (
    f"Multi-pathology prediction margin is narrow ({margin * 100:.1f}% margin, {confidence_pct:.1f}%
confidence). Clinician review recommended."
    if is_uncertain else None
)

# 6. Pathology-Specific Clinical Assessment & Recommendations
if pred_class == "Normal":
    diam_mm = 0.0
    primary_diag = "Normal Renal Parenchyma (Healthy)"
    severity = "Normale (Sain)"
    urgency = "Routine / None"
    recommendation = "Homogeneous renal parenchyma. No nephrolithiasis, cortical cyst, or suspicious
tissue mass detected. Normal routine follow-up."
elif pred_class == "Stone":
    primary_diag = "Kidney Stone Detected (Nephrolithiasis)"
    severity = "Moderate"
    urgency = "Urological Consultation"
    recommendation = "Renal calculus identified. Urological evaluation recommended for dimension
staging (ESWL vs Ureterscopy), hydration therapy, and metabolic workup."
elif pred_class == "Cyst":
    primary_diag = "Renal Cyst Detected (Bosniak I/II)"
    severity = "Benign Cystic Formation"
    urgency = "Periodic Ultrasound Follow-up"
    recommendation = "Well-defined fluid-filled cortical cyst consistent with Bosniak Category I/II.
Annual ultrasound monitoring advised to confirm size stability. No surgical intervention indicated."
elif pred_class == "Tumor":
    primary_diag = "Suspicious Renal Mass / Neoplasm Alert"
    severity = "Solid Tissue Mass (Potential Neoplasm)"
    urgency = "Urgent Specialist Evaluation"
    recommendation = "Solid heterogeneous renal lesion detected. Immediate diagnostic workup
required: Contrast-enhanced multiphasic CT / MRI and urgent urology/oncology consult for staging and
characterization."
else:
    primary_diag = f"{pred_class} Detected"
    severity = "Investigate"
    urgency = "Clinical Review"
    recommendation = "Clinical evaluation indicated."

# 7. Grad-CAM only for focal anomalies (Stone, Cyst, Tumor) – suppressed for Normal
if pred_class != "Normal":
    target_logit = output[0, pred_idx]
    target_logit.backward()

    cam_result = generate_pytorch_gradcam(
        model=master_model,
        tensor=tensor,
        class_idx=pred_idx,
        cleaned_pil=cleaned_pil,
        pred_class=pred_class,
        confidence_pct=confidence_pct,
        primary_diag=primary_diag,
        severity=severity,
        filename=filename
    )

```

```

        diam_mm = cam_result.get("estimated_diameter_mm", 0.0)
    else:
        cam_result = {
            "gradcam_image": None,
            "peak_coordinates": None,
            "estimated_diameter_mm": 0.0,
            "visualization_base64": None,
        }
        diam_mm = 0.0

    logger.info(
        f"[INFERENCE] Prediction: {pred_class} ({confidence_pct}%), "
        f"Gate: Passed ({kidney_prob * 100:.1f}%), Status: {triage_status}"
    )

    return {
        "task": AI_TASK_DESCRIPTION,
        "prediction": pred_class,
        "predicted_class": pred_class,
        "confidence": confidence_pct,
        "confidence_percent": confidence_pct,
        "probability": round(float(probs[pred_idx]), 4),
        "class_probabilities": class_probs,
        "has_stone": has_stone,
        "has_tumor": has_tumor,
        "has_cyst": has_cyst,
        "is_normal": is_normal,
        "stone_probability": class_probs.get("Stone", 0.0),
        "cyst_probability": class_probs.get("Cyst", 0.0),
        "tumor_probability": class_probs.get("Tumor", 0.0),
        "normal_probability": class_probs.get("Normal", 0.0),
        "primary_diagnosis": primary_diag,
        "is_uncertain": is_uncertain,
        "status": triage_status,
        "review_reason": review_reason,
        "model_version": "ConvNeXt-Tiny Master 4-Class SOTA + MobileNetV3 Gate",
        "gradcam_image": cam_result.get("gradcam_image"),
        "heatmap_url": cam_result.get("gradcam_image"),
        "peak_coordinates": cam_result.get("peak_coordinates"),
        "clinical_assessment": {
            "estimated_diameter_mm": diam_mm,
            "severity": severity,
            "urgency": urgency,
            "recommendation": recommendation,
        },
        "gate_evaluation": {
            "gate_passed": True,
            "kidney_confidence_percent": round(kidney_prob * 100, 2),
            "non_kidney_probability_percent": round(non_kidney_prob * 100, 2),
        },
        "telemetry": {
            "auto_masking": mask_telemetry,
            "classes_evaluated": CLASS_NAMES,
            "device": str(device)
        },
        "cleaned_scan_base64": cam_result.get("cleaned_scan_base64"),
        "visualization_base64": cam_result.get("visualization_base64")
    }, None, 200

```

```
except Exception as e:
```

```
logger.error(f"Inference execution error: {str(e)}", exc_info=True)
return None, f"Error during neural network inference: {str(e)}", 500
```

Code Source Intégral 2.6 — Orchestrateur d'inférence clinique (*app/services/inference_service.py*).

Analyse Ligne par Ligne de l'Orchestrateur

- **Lignes 30-55 (Étape 1 & 2 : Validation & Heuristiques)** : Lecture binaire du fichier, décodage PIL, validation géométrique et spectrale. Rejet immédiat avec code HTTP 400 ou 422 si le fichier est corrompu.
- **Lignes 57-84 (Étape 3 : Triage Gatekeeper)** : Évaluation de l'authenticité rénale par MobileNetV3. Si $P(\text{ext}\{\text{Non-Kidney}\}) \geq 70\%$, renvoie un message explicite invitant à téléverser un scanner abdominal conforme.
- **Lignes 86-98 (Étape 4 : Inpainting Telea)** : Débruitage automatique des annotations si le drapeau `auto_mask=True` est actif.
- **Lignes 100-130 (Étape 5 : Inférence 4 Classes & Marge d'Incertitude)** : Calcul des probabilités softmax. Si la marge entre les deux classes supérieures est inférieure à 5%, le statut passe en `review` pour alerter le praticien.
- **Lignes 132-157 (Étape 6 : Recommandations Médicales Spécifiques)** : Formulation de directives conformes aux guidelines urologiques (classification Bosniak pour kystes, lithotripsie pour calculs, bilan d'extension oncologique pour tumeurs).
- **Lignes 159-183 (Étape 7 : Activation Grad-CAM Conditionnelle)** : La carte de chaleur n'est générée que pour les anomalies focales (Stone, Cyst, Tumor) et reste neutralisée pour les examens normaux pour éviter toute confusion.

8. Endpoints REST : Inférence (predict.py) & Santé (health.py)

8.1 Contrôleur de Prédiction (app/routes/predict.py)

```

"""
Prediction route blueprint supporting dual-stage Gatekeeper & 4-Class Diagnostic.
"""
import logging
from flask import Blueprint, request, jsonify
from app.config import ALLOWED_EXTENSIONS, MAX_FILE_SIZE
from app.preprocessing.validator import allowed_file
from app.services.inference_service import process_and_predict

logger = logging.getLogger(__name__)

predict_bp = Blueprint('predict', __name__)

@predict_bp.route('/predict', methods=['POST', 'OPTIONS'])
@predict_bp.route('/api/predict', methods=['POST', 'OPTIONS'])
def predict():
    """Prediction endpoint for renal ultrasound and abdominal CT scans."""
    if request.method == 'OPTIONS':
        return "", 200

    logger.info("Received prediction request.")

    # Find file in common field names
    file_obj = None
    for field in ['image', 'scan', 'file', 'photo']:
        if field in request.files:
            file_obj = request.files[field]
            break

    if file_obj is None or file_obj.filename == '':
        if request.data:
            image_bytes = request.data
            filename = "scan.jpg"
        else:
            logger.warning("No image payload found in request.")
            return jsonify({"error": "No image provided. Please upload an image file under 'image' or 'scan'."}), 400
        else:
            filename = file_obj.filename
            if not allowed_file(filename):
                logger.warning(f"Invalid file extension: {filename}")
                return jsonify({"error": f"Invalid file format: '{filename}'. Allowed: {ALLOWED_EXTENSIONS}"})
            400

        try:
            image_bytes = file_obj.read()
            if len(image_bytes) > MAX_FILE_SIZE:
                logger.warning(f"File size exceeds limit: {len(image_bytes)} bytes")
                return jsonify({"error": f"File exceeds maximum size of {MAX_FILE_SIZE // (1024*1024)}MB"}),
            413

        except Exception as e:
            logger.error(f"Error reading file bytes: {str(e)}")
            return jsonify({"error": "Failed reading image stream."}), 400

```

```
# Auto mask option (default to True)
auto_mask_str = request.form.get("auto_mask", request.args.get("auto_mask", "true")).lower()
auto_mask = auto_mask_str in ["true", "1", "yes"]

result, error_msg, status_code = process_and_predict(image_bytes, filename=filename,
auto_mask=auto_mask)

if error_msg:
    is_gate = "Gatekeeper" in error_msg
    return jsonify({
        "error": error_msg,
        "gate_rejected": is_gate,
        "success": False
    }), status_code

return jsonify(result), status_code
```

Code Source Intégral 2.7 — Point d'entrée POST /predict (app/routes/predict.py).

8.2 Contrôleur de Santé & Monitoring (app/routes/health.py)

```
"""
Health check blueprint for KidneyVision AI microservice.
"""
from flask import Blueprint, jsonify
from app.inference.model_loader import get_master_model, get_gate_model, get_device
from app.config import CLASS_NAMES

health_bp = Blueprint('health', __name__)

@health_bp.route('/health', methods=['GET'])
@health_bp.route('/api/health', methods=['GET'])
def health_check():
    """Health endpoint reporting model availability, gatekeeper status, and hardware acceleration."""
    master = get_master_model()
    gate = get_gate_model()
    device = get_device()

    if master is None:
        return jsonify({
            "status": "unhealthy",
            "message": "Master ConvNeXt-Tiny model not loaded",
            "gpu_available": str(device).startswith("cuda")
        }), 503

    return jsonify({
        "status": "healthy",
        "service": "KidneyVision AI Multi-Pathology Diagnostic Service",
        "models": {
            "master_model": "ConvNeXt-Tiny (Master 4-Class SOTA)",
            "gatekeeper_model": "MobileNetV3-Small (Kidney Gate)",
            "gate_active": gate is not None
        },
        "supported_classes": CLASS_NAMES,
        "device": str(device),
        "gpu_available": str(device).startswith("cuda")
    }), 200
```

Code Source Intégral 2.8 — Endpoint de monitoring GET /health (app/routes/health.py).

9. Synthèse Technique du Microservice Flask

Le microservice Flask de KidneyVision AI démontre comment isoler des modèles de Deep Learning complexes au sein d'une API autonome, robuste, auto-documentée et résiliente, prête à être intégrée au système d'information hospitalier via le backend Laravel.